

CS 2430 – Project 0

Programming Paradigms and Project Workflow

Name: Connor Callahan

Course: CS 2430

Semester: Summer 2026

Date: May 26, 2026

Programming Paradigms

1 Object-Oriented Programming

Object-oriented programming centers on building software from self-contained units called objects each combining its own data with the behaviors that act on it. Its four foundational principles are encapsulation, abstraction, inheritance and polymorphism. Rather than following one continuous sequence of instructions a program built this way is a collection of cooperating objects.

OOP shines when a problem naturally maps to multiple real-world entities that each hold state and perform actions particularly when those entities form parent-child relationships. A college course registration system with Student, Course and Instructor classes is a good example because each entity manages its own data independently and a specialized type like GraduateStudent can build on Student through inheritance without duplicating any existing logic. Applications that grow large and involve many interacting parts benefit the most from this structure.

Where OOP falls short is small scripts or straightforward data-processing tasks. Building class hierarchies in those situations just adds complexity without any real organizational benefit.

Java is the standard language for this paradigm since all executable code must live inside a class which enforces the OOP model at every level. Features like interfaces, abstract classes and access modifiers directly support all four core principles. A good starting point is the Oracle Java Tutorials at <https://docs.oracle.com/javase/tutorial/java/concepts/> and I have used this paradigm in previous courses so I am fairly comfortable with it.

2 Procedural Programming

Procedural programming organizes a program as an ordered series of instructions grouped into reusable functions. Data and behavior remain separate where variables store information and functions process it and the programmer directly controls execution flow from start to finish.

This style works well for problems that follow a straightforward step-by-step sequence and do not require complex relationships between entities. A command-line calculator that accepts two numbers and an operator then calls the appropriate function for each operation is a good fit because the task is a simple

linear workflow with no need for objects or class hierarchies. Automation scripts and low-level systems programming are other natural applications.

Where it breaks down is in large applications. As a codebase scales up any function can read or modify shared data which makes it increasingly hard to track down the source of bugs.

C++ is a strong fit for this paradigm because it allows code to be written outside of classes entirely. Its close relationship with hardware memory and its function-based design make it common in operating systems and embedded development where fine-grained control over resources matters. More information can be found at <https://cplusplus.com/doc/tutorial/> and like OOP I have used this paradigm in previous courses.

3 Functional Programming

Functional programming treats computation as a series of pure function evaluations applied to data that never changes. Functions are first-class values that can be passed around and returned like any other data and the primary goal is to eliminate hidden side effects so program behavior is easier to reason about.

This paradigm works really well for data-transformation pipelines and concurrent systems. Because pure functions don't share any mutable state they can safely run in parallel without risking data-race conditions. A data analysis tool that filters a dataset and computes aggregate statistics is a good example, each processing step is a clean transformation with no shared state to worry about which makes every step easy to test on its own.

Where it falls short is anything that involves heavily stateful interfaces. Something like a GUI application where controls need to continuously track and respond to user input doesn't map naturally to this style at all.

Haskell is the standard language associated with this paradigm. It enforces functional purity at the language level so accidental mutation of shared state is basically impossible and its strong static type system lets you rule out illegal program states at compile time. A good starting resource is Lipovaca's "Learn You a Haskell for Great Good!" at <https://learnyouahaskell.com/> and my current familiarity with this paradigm is pretty limited, I have only read about it and have no hands-on experience yet.

Paradigms vs. Architectural Patterns vs. Design Patterns

1 Distinguishing the Three Levels

A programming paradigm is a fundamental philosophy of computation that shapes how a developer thinks about and structures an entire program at the language level. It determines which constructs are available and guides how the developer reasons about problems throughout the whole codebase.

An architectural pattern is a high-level blueprint for dividing a system into major components defining how they communicate and distributing responsibilities among them. These decisions influence

deployment, scalability and team organization and they are made before a single line of code is written.

A design pattern is a proven reusable solution to a specific recurring problem that appears within a codebase. It operates at a smaller scope than an architectural pattern and describes how a handful of classes or functions should interact to address one well-defined challenge. The foundational reference is Gamma et al. 1994 which catalogued 23 patterns grouped into creational, structural and behavioral categories.

Put simply a paradigm governs the style you think in, an architectural pattern governs how an entire system is divided and a design pattern governs how a specific recurring problem is solved inside that system.

2 Architectural Patterns

The client-server pattern splits a system into clients that make requests and servers that fulfill them communicating through a defined network protocol. This solves the problem of making shared data reliably available to many users without requiring each user to maintain their own local copy. The most familiar example is the web browser and web server relationship though the same structure also underlies email clients and multiplayer games.

Layered architecture organizes a system into horizontal tiers commonly presentation, business logic and data access where each tier only communicates with the one immediately above or below it. This manages complexity in large systems by isolating changes so that a modification in one layer does not ripple into unrelated layers. The majority of enterprise web applications follow a three-tier arrangement consisting of a frontend, a business logic API and a relational database.

Microservices breaks an application into small independently deployable services each owning its own data and exposing a well-defined API with services communicating over lightweight protocols. This solves scaling and deployment bottlenecks by allowing a single high-demand service to be scaled on its own without redeploying the entire application. Netflix and Amazon are frequently cited examples of this approach.

A solid resource covering all three of these patterns is Richards and Ford's "Fundamentals of Software Architecture" from O'Reilly Media at <https://www.oreilly.com/library/view/fundamentals-of-software/9781492043447/>

3 Design Patterns

The observer pattern addresses the problem of multiple objects needing to react when another object changes state without the subject needing to know who is watching. The subject keeps a list of observers and notifies all of them through a shared interface whenever its state updates which means observers can be added or removed at runtime without touching the subject at all. A weather dashboard where multiple display panels refresh automatically when new sensor readings arrive is a common illustration of this.

The factory method pattern addresses the problem of a class needing to produce objects without being tied to a specific concrete type at compile time. A creation interface is defined but subclasses or configuration determine which concrete class is actually instantiated which means new product types can

be introduced without modifying existing code. A cross-platform UI toolkit that returns the appropriate button type for each operating system is a typical example.

The singleton pattern addresses the problem of a system needing exactly one instance of a class such as a logger or configuration manager that is accessible from anywhere in the application. The class holds a private static reference to its sole instance and exposes it through a static accessor while keeping its constructor private to prevent additional instantiation. An application-wide logging service is the standard illustration of this pattern.

A good reference for all three of these is the summary guide at <https://refactoring.guru/design-patterns> which is based on Gamma, Helm, Johnson and Vlissides "Design Patterns: Elements of Reusable Object-Oriented Software" from Addison-Wesley 1994.

Learning Plan and Reflection

Most of my programming experience has been in object-oriented and procedural styles because those are the paradigms covered in introductory courses. Functional programming was new to me going into this assignment and researching it was genuinely interesting but my actual focus this semester is getting better at looking at a problem and building a real plan before touching any code. It does not matter much what language that ends up being in, what I want to improve is understanding the logic and structure behind each step well enough that the implementation almost follows naturally.

The reason that matters to employers is not just that you know more languages or patterns its that you can walk into an unfamiliar codebase or problem and not freeze up. Being able to break something down systematically and recognize structure you have seen before means you spend less time confused and more time actually building things. That is the skill I want to be developing this semester.

Repository and Project Workflow

Repository URL

<https://github.com/SaltyCrew19/cs2430-projects>

Repository Structure

```
cs2430-projects/  
├── README.md  
├── .gitignore  
├── docs/  
├── project1/  
│   ├── src/  
│   └── docs/  
└── project2/
```

```
| ├── src/
| └── docs/
└── project3/
    ├── src/
    └── docs/
└── project4/
    ├── src/
    └── docs/
```

Each project folder is self-contained with dedicated source and docs subfolders so reviewing or submitting any individual project does not require navigating the whole repository. The root README describes the course, the semester and the purpose of each folder and notes that Project 0 is a setup assignment while Projects 1 through 4 get added as the semester goes on. The .gitignore excludes compiled binaries, IDE configuration folders and operating system-generated files.

For quality I tend to write new code in shorter focused sessions and save the longer blocks for debugging. When something breaks I reach for the debugger first and fall back to print statements when I cant. A habit I want to break is forgetting what I already did and having to debug my way back to it so I will commit to Git at the end of every session with a message that actually explains what changed and why so I always have a trail to follow.

For maintainability every function will have a comment covering its purpose, parameters and return value. Right now my version of planning is usually just dropping todo comments into the code as I go which works but means I am figuring things out mid-implementation rather than before. For each of Projects 1 through 4 I want to write an actual design note covering the algorithm, data structures and edge cases before writing any real code. It does not need to be formal but it needs to exist before I start.

My available work blocks are Tuesday 6:00 PM to 8:00 PM, Thursday 7:00 PM to 9:00 PM and Saturday 10:00 AM to 12:00 PM. I work weekdays until 6:00 so those are the slots I have.

References

Gamma, E., Helm, R., Johnson, R., and Vlissides, J. "Design Patterns: Elements of Reusable Object-Oriented Software." Addison-Wesley, 1994.

Lipovaca, M. "Learn You a Haskell for Great Good!" No Starch Press, 2011. <https://learnyouahaskell.com/>

Oracle. "The Java Tutorials: Object-Oriented Programming Concepts."
<https://docs.oracle.com/javase/tutorial/java/concepts/>

cplusplus.com. "C++ Language Tutorial." <https://cplusplus.com/doc/tutorial/>

Richards, M. and Ford, N. "Fundamentals of Software Architecture." O'Reilly Media, 2020.
<https://www.oreilly.com/library/view/fundamentals-of-software/9781492043447/>

Refactoring.Guru. "Design Patterns." <https://refactoring.guru/design-patterns>